

Coding for Artificial Intelligence

Structured Data Representation

Problem Set 5.2

Dictionaries, Sets, and Text Normalization

Lab Project: The AI Social Media Auditor

Course	Coding for AI
Author	Dr. Prateek, Assistant Professor, School of AI
Format	In-class practical, 1 hour 30 minutes total, inclusive of write-up and submission
Total Marks	100

Read this document in full before you begin writing code.

Contents

1	Objective	2
2	Task 1: Word Frequency Counter (35 minutes)	2
2.1	Requirements	2
2.2	Critical Fix: Text Normalisation	2
2.3	Example	3
3	Task 2: The Follower Deduplicator (35 minutes)	3
3.1	Requirements	3
3.2	Clarification: “Difference” Is Directional	3
3.3	Example	4
4	Technical Constraints	4
5	Edge Cases to Handle	4
6	Suggested Time Allocation	4
7	Deliverables	5
	Academic Integrity	5

1 Objective

Problem Set 5.1 introduced nested dictionaries and sets for representing structured agent state. This practical applies the same two data structures to a different, very common class of problem: **auditing unstructured text and user data**. You will build a small tool with two independent halves — a word-frequency counter and a follower deduplicator — that together form “The AI Social Media Auditor.”

By the end of this practical you should be able to:

- (i) use `setdefault` to build a frequency count from scratch,
- (ii) recognise *why* raw text must be normalised before counting words, and
- (iii) use set intersection, difference, and symmetric difference to compare two collections of unique identifiers.

Time budget. This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 6.

2 Task 1: Word Frequency Counter (35 minutes)

2.1 Requirements

1. Initialise an empty dictionary called `word_counts`.
2. Take a paragraph of text and use a loop (`for word in text.split():`) to iterate through every word.
3. Use `setdefault` to increment the count for each word:

```
1 word_counts.setdefault(word, 0)
2 word_counts[word] += 1
```

2.2 Critical Fix: Text Normalisation

If you count words exactly as `text.split()` produces them, your counts will be **wrong**. Consider the words "AI", "ai", and "AI," (with a trailing comma) — to a human reader these are clearly the same word used three times, but to Python they are three different dictionary keys, because string equality is exact and case-sensitive, and punctuation characters are just more characters. Before counting each word, you must:

1. convert it to lower case with `.lower()`, and
2. strip surrounding punctuation with `.strip(string.punctuation)` (this requires `import string` at the top of your file).

```
1 import string
2
3 word_counts = {}
4 for raw_word in text.split():
5     cleaned_word = raw_word.lower().strip(string.punctuation)
6     if not cleaned_word:
7         continue
8     word_counts.setdefault(cleaned_word, 0)
9     word_counts[cleaned_word] += 1
```

Why the `if not cleaned_word: continue` line matters. A token that is *only* punctuation — for example, a lone “---” used as a dash between sentences — becomes an empty string once stripped. An empty string is a legal but meaningless dictionary key; skipping it keeps your word counts limited to actual words.

2.3 Example

```
1 text = ("AI is changing the world. ai is powering new tools , and AI
    is helping researchers learn faster. The world loves AI, and the
    world depends on it more each year.")
```

After normalisation, this produces (elements shown alphabetically for readability; your dictionary’s natural order may differ):

```
1 ai: 4      is: 3      the: 3      world: 3      and: 2
2 changing: 1 depends: 1 each: 1  faster: 1  helping: 1
3 it: 1      learn: 1   loves: 1   more: 1    new: 1
4 on: 1      powering: 1 researchers: 1 tools: 1  year: 1
```

Note that “AI”, “ai”, and “AI,” are all counted together as a single key, “ai”, with a total of 4 — this is the direct result of the normalisation step in Section 2.2. Without it, your dictionary would contain three separate, smaller counts instead of one correct one.

3 Task 2: The Follower Deduplicator (35 minutes)

3.1 Requirements

1. Given two lists of “User IDs,” representing followers from two different platforms, convert each list into a **set**.
2. Find the **intersection** to see which users follow the AI on both platforms.
3. Identify which users are unique to **each** platform.

3.2 Clarification: “Difference” Is Directional

Set difference is not symmetric: `set_a - set_b` and `set_b - set_a` are generally two different sets. To identify users unique to *each individual platform* (and know which platform each one belongs to), you need **both** directions:

```
1 set_a = set(platform_a_followers)
2 set_b = set(platform_b_followers)
3
4 both_platforms = set_a & set_b          # intersection
5 only_a = set_a - set_b                  # unique to Platform A
6 only_b = set_b - set_a                  # unique to Platform B
```

If you only need a single combined set of “everyone who follows on exactly one platform, regardless of which,” Python provides this directly as the **symmetric difference** operator, `^`, which is equivalent to `only_a | only_b` but computed in one step:

```
1 either_only = set_a ^ set_b
```

Decide which of these your report actually needs — the two directional differences, or the single symmetric difference — based on whether you need to know *which* platform each unique follower came from.

3.3 Example

```
1 platform_a_followers = ["u101", "u102", "u103", "u104", "u105"]
2 platform_b_followers = ["u103", "u104", "u106", "u107"]
```

Quantity	Result
Follows on both platforms (&)	{"u103", "u104"}
Unique to Platform A (-)	{"u101", "u102", "u105"}
Unique to Platform B (-)	{"u106", "u107"}
Unique to exactly one platform (^)	{"u101", "u102", "u105", "u106", "u107"}

As in Problem Set 5.1, Python does not guarantee the order in which set elements print; only the membership of each set is graded, not the display order.

4 Technical Constraints

- **PEP 8 Compliance:** `snake_case` for all variables; one space around every operator.
- **Do not use** `collections.Counter` for Task 1, even though it would solve word-frequency counting in one line. The point of this exercise is to practise `setdefault` directly; you are welcome to redo the task with `Counter` afterwards, for your own interest, once your manual version is working and submitted.
- **Docstring Quality:** a professional header docstring describing both tasks and your author information.

5 Edge Cases to Handle

1. **An empty paragraph** (`text = ""`). Confirm `text.split()` produces an empty list and your loop simply does not execute, leaving `word_counts` as `{}` rather than raising an error.
2. **A word that is punctuation only**, e.g., a stray `--` token. Confirm your `if not cleaned_word:` check discards it rather than adding an empty-string key.
3. **Two follower lists with no overlap at all**. Confirm `both_platforms` correctly comes out as an empty set, and that `only_a` and `only_b` together account for every user in both original lists.
4. **Two identical follower lists**. Confirm `only_a` and `only_b` are both correctly empty, and `both_platforms` equals the full set of users.

6 Suggested Time Allocation

Phase	Description	Duration
Task 1	Word Frequency Counter, including normalisation	35 minutes
Task 2	Follower Deduplicator	35 minutes
Testing	Edge cases from Section 5	10 minutes
Documentation & Submission	Header comment, final write-up	10 minutes
Total		90 minutes

7 Deliverables

1. **Source Code:** A single file named `social_media_auditor.py` implementing both tasks.
2. **Header Documentation:** A comment block at the top of your file explaining, in your own words, why raw text must be normalised before counting, and the difference between `set_a - set_b` and `set_a ^ set_b`.
3. **Submission Note:** A short paragraph (100–150 words) stating, for your own test paragraph and follower lists, what your program actually output, and confirming it matches your hand-worked expectation.

Academic Integrity

You may discuss approaches with peers during the supervised practical session, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute's academic integrity policy.